UNIVERSITY OF ASIA PACIFIC
DEPARTMENT OF CSE

Course Code: CSE 402.

Course Title: Operating System Lab.

Experiment No: 1

Experiment Title: Implementation of First Come First Serve (FCFS) CPU Scheduling Algorithm**.**

Date of Submission : 8/3/2026

Submitted By:

Name : SAZID ABDULLAH

Student Id : 23101146

Semester : 4^{4h} Year 1st Semester

Section : C-2

Submitted To

Aita Rahman Orthi

Lecturer , UAP

Problem Statement:

Implement FCFS CPU Scheduling for the following scenario where No of processes = 5 and Process informations are-

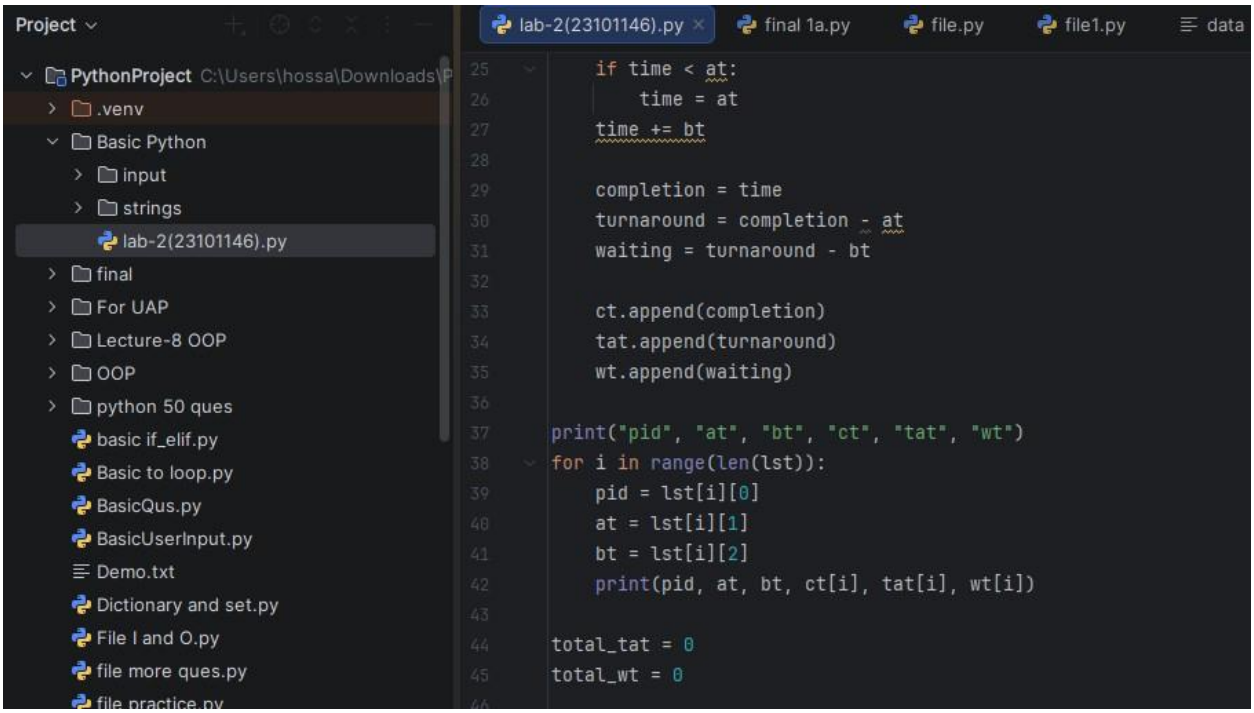
P id	AT	BT
P0	3	1
P1	5	3
P2	2	2
P3	1	2
P4	6	3

Calculate CT, TAT, WT, AVG TAT and AVG WT. Also represent the process execution sequence.

Source Code:

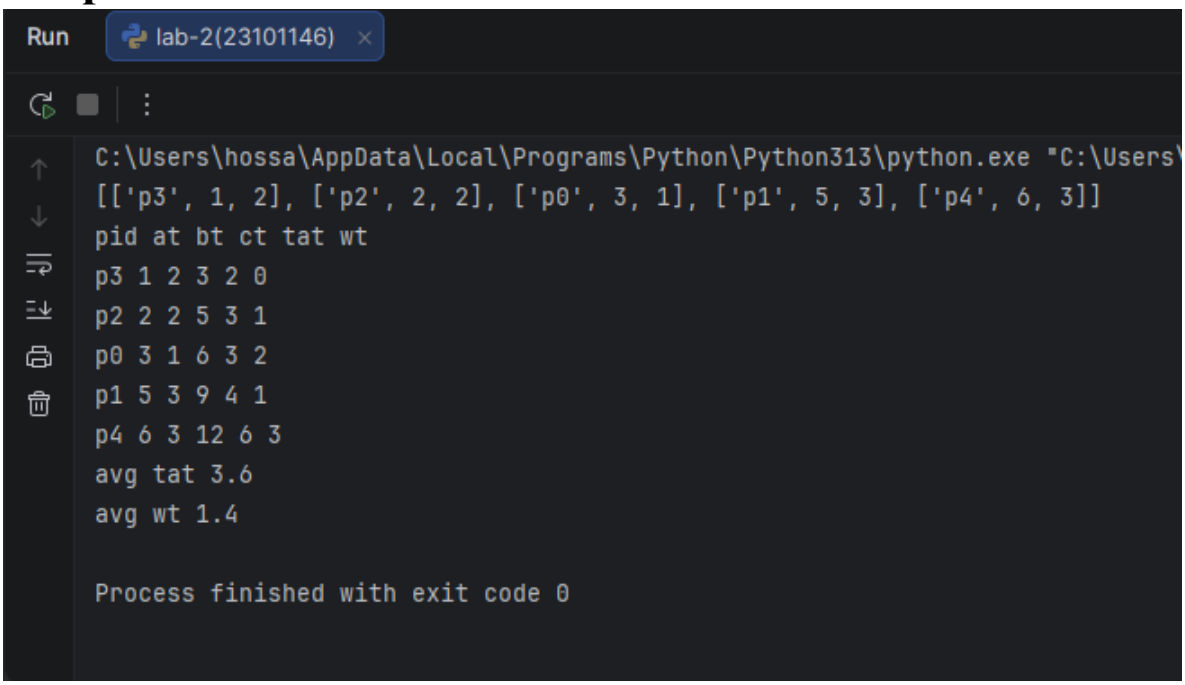
lab-2(23101146).py | final 1a.py | file.py | file1.py

```python
import numpy as np


lst = [
    ["p0", 3, 1],
    ["p1", 5, 3],
    ["p2", 2, 2],
    ["p3", 1, 2],
    ["p4", 6, 3],

]

lst.sort(key=lambda x :{__getitem__} : x[1])
print(lst)

ct = []
tat = []
wt = []

time = 0

for p in lst:
    pid, at, bt = p[0], p[1], p[2]

```




```
46
47      for i in range(len(lst)):
48          total_tat = total_tat + tat[i]
49          total_wt = total_wt + wt[i]
50
51      avg_tat = total_tat / len(lst)
52      avg_wt = total_wt / len(lst)
53
54      print("avg tat", avg_tat)
55      print("avg wt", avg_wt)
```

Output:



Discussion:

This experiment focused on implementing the First Come First Serve (FCFS) CPU scheduling algorithm using Python. Each process was represented with three key attributes — Process ID, Arrival Time (AT), and Burst Time (BT) — and stored in a list. Since FCFS executes processes strictly in the order they arrive, the list was first sorted by arrival time before any scheduling logic was applied.

The program also handled CPU idle time properly. Before starting a process, it checked whether the CPU's current time was behind the process's arrival time. If so, the CPU was forced to wait (go idle) until that process actually arrived, ensuring the timing calculations stayed accurate. Once a process started execution, it ran to completion without being interrupted, since FCFS is a non-preemptive algorithm.

After each process finished, three important metrics were calculated:

- **Completion Time (CT)** – when the process finishes
- **Turnaround Time (TAT)** – CT − AT
- **Waiting Time (WT)** – TAT − BT

At the end, the average TAT and average WT were computed to measure how efficiently the algorithm performed overall.

From the output, it was clear that all processes ran exactly in their arrival order, with no preemption at any stage. While this makes FCFS very predictable and easy to trace, it also exposed a major weakness: if a process with a large burst time arrives first, every process behind it is forced to wait longer than necessary — even if those processes have very small burst times. This problem, known as the **Convoy Effect**, can significantly increase average waiting time and hurt system responsiveness.

Conclusion:

This experiment successfully achieved its goal of implementing and analyzing the FCFS CPU scheduling algorithm in Python. The program correctly sorted processes by arrival time, computed CT, TAT, and WT for each process, and calculated the average TAT and WT to evaluate performance.

Through this implementation, the core working principle of FCFS became clear — processes are scheduled purely based on arrival order, with no consideration of burst time or priority. This makes FCFS simple, fair in terms of arrival sequence, and easy to code, which is why it's often used as an introductory example in operating systems courses.

However, the experiment also highlighted FCFS's main drawback: poor performance when burst times vary significantly, due to the Convoy Effect. In real-world systems where responsiveness and fairness across varying job sizes matter, algorithms like **Shortest Job First (SJF)** or **Round Robin (RR)** tend to perform better, since they either prioritize shorter jobs or give every process a fair time slice. Overall, this experiment gave a solid practical understanding of how basic CPU scheduling works and why more advanced algorithms are needed in real operating systems.